Vivekanand Education Society's Institute of Technology

An Autonomous Institute Affiliated to University of Mumbai

Hashu Advani Memorial Complex, Collector Colony, Chembur East, Mumbai – 400074

Department of Artificial Intelligence and Data Science

A Major Project Report on

Smart Mall Parking Management System

Submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Engineering

in

Artificial Intelligence and Data Science

Submitted by

[Piyush Kurseja] – [ROLL NO:- 65]

Under the Guidance of

[Anil Ahir]

Project Guide

Academic Year 2026–2027

Certificate

This is to certify that the project entitled **"Smart Mall Parking Management System"** has been carried out by [Piyush Kurseja] under the guidance of [Anil Ahir] in partial fulfilment of the requirements for the award of the degree of Bachelor of Engineering in Artificial Intelligence and Data Science during the academic year 2026–2027.

[GUIDE NAME]	[HOD NAME]	[PRINCIPAL NAME]
Project Guide	Head of Department	Principal

Date: 17/10/2026 Place: Mumbai

Declaration

We hereby declare that the project entitled **"Smart Mall Parking Management System"** is our original work carried out under the guidance of [Anil Ahir]. The work presented in this report has not been submitted elsewhere for the award of any degree or diploma, except where due acknowledgement has been made. We take responsibility for accurately describing the implementation and acknowledging the sources, libraries, models, and tools used.

Name	**Signature**
[Piyush Kurseja]	_____

Acknowledgement

We would like to express our sincere gratitude to our project guide, [Anil Ahir], for the guidance, suggestions, and encouragement provided throughout the development of our project, **Smart Mall Parking Management System**. We are thankful to the Head of the Department and faculty members of the Department of Artificial Intelligence and Data Science for their academic support. We also acknowledge the developers and contributors of Python, OpenCV, Ultralytics YOLO, NumPy, and other open-source technologies used in this project. Finally, we thank our families and friends for their motivation and support.

[Piyush Kurseja]

Abstract

Parking management is a significant challenge in shopping malls and other commercial establishments, where visitors may spend time searching for available parking spaces. The Smart Mall Parking Management System is a computer-vision-based prototype designed to estimate parking-space occupancy from recorded video footage. The system is implemented using Python, OpenCV, and the Ultralytics YOLO object-detection model. It processes recorded parking videos, detects cars, and compares detected vehicle bounding boxes with predefined parking-space regions stored in JSON configuration files. It estimates whether each configured space is occupied or available and calculates parking statistics for three configured floors. A dashboard presents total, occupied, and available parking-space counts and displays a recommendation based on estimated occupancy. The prototype demonstrates how object detection and video processing can support parking monitoring. Its performance depends on video quality, camera viewpoint, configured-region accuracy, and detection reliability. The current implementation uses recorded videos and does not claim automatic live CCTV integration or production deployment. Future work may include authorised live-camera integration, improved validation, persistent storage, and a web dashboard.

Keywords: Smart Parking, Computer Vision, YOLO, OpenCV, Vehicle Detection, Occupancy Estimation, Parking Dashboard.

Contents

Certificate i

Declaration ii

Acknowledgement iii

Abstract iv

List of Figures viii

List of Tables ix

1 Introduction 1

1.1 Background . . . 1
1.2 Problem Statement . . . 1
1.3 Motivation . . . 1
1.4 Objectives . . . 1
1.5 Scope . . . 2
1.6 Contributions . . . 2
1.7 Organization of the Report . . . 2

2 Literature Review 3

2.1 Introduction . . . 3
2.2 Existing Parking-Management Approaches . . . 3
2.3 Computer Vision for Vehicle Detection . . . 3
2.4 YOLO Object Detection . . . 3
2.5 OpenCV-Based Video Processing . . . 3
2.6 Parking-Space Occupancy Estimation . . . 4
2.7 Comparison of Approaches . . . 4
2.8 Research Gap . . . 4
2.9 Summary . . . 4

3 Proposed Methodology 5

3.1 Overview 5
3.2 Hardware Requirements 5
3.3 Software Requirements 5
3.4 Proposed Approach 6
3.5 Algorithms and Models 6
3.5.1 YOLO-Based Car Detection 6
3.5.2 Parking-Region Matching 6
3.5.3 Occupancy Stability 7
3.5.4 Available-Space Calculation 7
3.6 Data Collection and Preprocessing 7
3.7 Evaluation Metrics 7
3.8 Summary 7

4 System Design **8**
4.1 System Overview 8
4.2 System Architecture 8
4.2.1 Architecture Components 9
4.3 System Workflow 9
4.4 Data Flow 11
4.5 Parking-Space Configuration Design 11
4.6 Module Description 11
4.7 Design Considerations and Limitations 12
4.8 Summary 12

5 Implementation **13**
5.1 Development Environment 13
5.2 Project Structure 13
5.3 Video Input 13
5.4 Vehicle Detection 14
5.5 Parking-Space Configuration 14
5.6 Occupancy Estimation 14
5.7 Statistics Calculation 14
5.8 Parking Recommendation 15
5.9 Dashboard 15

5.10 User Interface 15
5.10.1 First Floor Parking Visualisation 15
5.10.2 Second Floor Parking Visualisation 16
5.10.3 Third Floor Parking Visualisation 16
5.10.4 Parking Management Dashboard 16
5.11 Integration 17
5.12 Security and Privacy 17
5.13 Summary 17

6 Results and Discussion **18**
6.1 Experimental Setup 18
6.2 Test Cases 18
6.3 Observed Results 19
6.4 Quantitative Evaluation 19
6.5 Discussion 19
6.6 Limitations 19
6.7 Summary 19

7 Conclusion and Future Scope **20**
7.1 Conclusion 20
7.2 Limitations 20
7.3 Future Scope 20
7.4 Final Remarks 21

References **22**

A Additional Project Information **23**
A.1 Execution Checklist 23
A.2 Screenshots and Evidence 23
A.3 Team Contributions 23

List of Figures

4.1 System architecture of the Smart Mall Parking Management System . 8
4.2 Processing workflow for parking occupancy estimation 10

5.1 Parking Occupancy Detection on the First Floor 15
5.2 Parking Occupancy Detection on the Second Floor 16
5.3 Parking Occupancy Detection on the Third Floor 16
5.4 Smart Mall Parking Management System Dashboard 17

List of Tables

2.1 General comparison of parking-monitoring approaches 4

3.1 Hardware used for development and testing 5

3.2 Software requirements 5

4.1 System modules and responsibilities 12

6.1 Functional test cases 18

Chapter 1

Introduction

1.1 Background

Parking management is an important concern in shopping malls and commercial establishments. During peak hours, drivers may spend considerable time searching for vacant spaces, causing congestion and inconvenience. Artificial Intelligence and Computer Vision can analyse video frames and estimate parking occupancy. This project uses Python, OpenCV, and Ultralytics YOLO to detect cars in recorded videos and compare detections with predefined parking-space regions. The estimated status is presented through a video interface and a dashboard for three configured floors.

1.2 Problem Statement

Manual identification of available parking spaces can be time-consuming, particularly when a parking facility has multiple floors. A monitoring system should detect cars, associate detections with parking regions, estimate available spaces, and present the information clearly. This project develops a video-based prototype that estimates configured parking-space occupancy and recommends an available space.

1.3 Motivation

The project is motivated by the effort involved in locating parking spaces and by the opportunity to demonstrate Computer Vision in a practical setting. It combines object detection, video processing, region matching, and dashboard visualisation.

1.4 Objectives

1. Detect cars in recorded parking videos using YOLO.
2. Configure parking-space regions using JSON files.
3. Estimate the status of individual parking spaces.
4. Calculate total, occupied, and available spaces for three configured floors.

5. Display vehicle detections and parking statistics.

6. Recommend an available space based on estimated occupancy.

1.5 Scope

The project processes recorded videos and predefined parking-space regions. It includes vehicle detection, occupancy estimation, floor-wise statistics, a dashboard, and a basic recommendation. Automatic access to private CCTV, production deployment, live multi-camera synchronisation, mobile application support, and persistent database storage are outside scope unless separately implemented and tested.

1.6 Contributions

The project integrates YOLO-based car detection, JSON-based parking-space configuration, geometric occupancy estimation, floor-wise statistics, dashboard visualisation, and a basic availability recommendation.

1.7 Organization of the Report

Chapter 2 reviews relevant approaches. Chapter 3 describes methodology and requirements. Chapter 4 presents system design. Chapter 5 explains implementation. Chapter 6 describes testing and results. Chapter 7 concludes the report and presents future scope.

Chapter 2

Literature Review

2.1 Introduction

Parking-management systems use manual observation, physical sensors, entry/exit monitoring, and camera-based analysis. This chapter reviews the concepts relevant to the proposed prototype.

2.2 Existing Parking-Management Approaches

Manual monitoring requires continuous attention, while sensor-based approaches may require installation and maintenance of hardware. Camera-based systems analyse images or video to infer vehicle presence. These systems can cover multiple spaces in a frame but are affected by camera position, lighting, occlusion, and detection quality.

2.3 Computer Vision for Vehicle Detection

Computer Vision extracts information from images and video. Vehicle detectors identify vehicles and represent them with bounding boxes. These coordinates can be compared with predefined parking regions to estimate whether a space is occupied.

2.4 YOLO Object Detection

You Only Look Once (YOLO) is an object-detection approach that predicts object locations and classes in images. The Ultralytics implementation provides an interface for applying trained YOLO models to video frames. In this project, YOLO detects cars and supplies coordinates for occupancy estimation.

2.5 OpenCV-Based Video Processing

OpenCV provides tools for reading video files, processing frames, drawing bounding boxes, displaying images, and handling keyboard input. It is used to read recorded videos, visualise detections, draw parking-space status, and render the dashboard.

2.6 Parking-Space Occupancy Estimation

Vehicle detection alone does not identify which parking space is occupied. Detected coordinates must be compared with configured regions. Intersection-over-union and point-in-polygon checks can help associate detections with spaces. A stability mechanism can reduce rapid status changes caused by temporary detection fluctuations.

2.7 Comparison of Approaches

Table 2.1: General comparison of parking-monitoring approaches

Approach	**Strength**	**Limitation**
Manual monitoring	Simple to start	Requires continuous observation
Space sensors	Can report sensor state	Requires installation and maintenance
Video-based detection	Can analyse multiple spaces in a frame	Affected by occlusion, lighting, and camera angle
Proposed prototype	Combines detections, regions, and dashboard	Uses recorded video and needs region configuration

2.8 Research Gap

Vehicle detection alone is not sufficient for parking management. Detections must be associated with individual spaces and converted into understandable availability information. The proposed project integrates detection, configured parking regions, occupancy estimation, floor-wise counts, and a dashboard using recorded videos.

2.9 Summary

This chapter reviewed traditional parking approaches, Computer Vision, YOLO, OpenCV, and occupancy estimation. Add verified research papers and cite them in the final literature survey before submission.

Chapter 3

Proposed Methodology

3.1 Overview

The system processes recorded video for each configured floor. YOLO detects cars in frames; the occupancy module compares bounding boxes with regions loaded from JSON files. The system calculates parking statistics and displays the video or dashboard.

3.2 Hardware Requirements

Record the actual computer specifications used for development and testing.

Table 3.1: Hardware used for development and testing

Component	**Actual specification**
Processor	[Enter processor model]
RAM	[Enter installed RAM]
GPU	[Enter GPU or write Not used]
Operating system	[Enter operating system]
Storage	[Enter relevant specification]

3.3 Software Requirements

Table 3.2: Software requirements

Software	**Purpose**
Python	Main programming language
Ultralytics YOLO	Car detection
OpenCV	Video processing and visualisation
NumPy	Coordinate and numerical operations
JSON	Parking-space configuration
Visual Studio Code or another IDE	Source-code editing and execution

3.4 Proposed Approach

1. Load YOLO model weights.
2. Load parking-space configurations for the selected floor.
3. Open the corresponding recorded video.
4. Read a video frame.
5. Detect cars using YOLO.
6. Compare bounding boxes with configured parking regions.
7. Update estimated occupancy.
8. Calculate occupied and available counts.
9. Display annotated video or dashboard.
10. Continue until the user exits.

3.5 Algorithms and Models

3.5.1 YOLO-Based Car Detection

YOLO returns bounding boxes, class identifiers, and confidence scores. The implementation filters predictions to the car class defined by the selected model. The confidence threshold affects which predictions are accepted.

3.5.2 Parking-Region Matching

For a detected bounding box B and parking region P, intersection-over-union is:

$$\mathrm{IoU}(B, P) = \frac{|B \cap P|}{|B \cup P|}.$$

A sufficient overlap or point-in-polygon test can associate a detection with a region. Use the thresholds present in the actual implementation.

3.5.3 Occupancy Stability

A per-space stability counter can reduce rapid changes across adjacent frames. The counter is updated according to detections and the implementation's configured threshold.

3.5.4 Available-Space Calculation

$$N_{\text{available}} = N_{\text{total}} - N_{\text{occupied}}.$$

3.6 Data Collection and Preprocessing

Inputs include recorded parking videos and manually configured parking regions. Coordinates must align with the video viewpoint and resolution. Videos should be checked for readability. Quantitative accuracy evaluation requires labelled occupancy data.

3.7 Evaluation Metrics

Use actual measurements only. Potential metrics include processing speed (frames per second), detection confidence, and occupancy accuracy, precision, and recall where ground truth is available. Do not include unmeasured performance claims.

3.8 Summary

The methodology combines recorded-video input, YOLO car detection, JSON parking regions, occupancy estimation, and dashboard output.

Chapter 4

System Design

4.1 System Overview

The Smart Mall Parking Management System is a computer-vision prototype that estimates parking-space occupancy from recorded parking videos. The system uses Python for application logic, OpenCV for video-frame handling and visualisation, and the Ultralytics YOLO model for vehicle detection. Parking-space regions are configured using JSON files. The detections are compared with the configured regions to estimate whether each space is occupied or available. The resulting counts are presented through a dashboard.

The prototype is organised into input, detection, occupancy-estimation, statistics, and presentation stages. It processes the selected video rather than claiming direct access to private mall CCTV feeds. The three configured parking floors are represented by their respective floor visualisations. The image named `4th Floor` is used as the dashboard screenshot and does not represent a fourth parking floor.

4.2 System Architecture

Figure 4.1 illustrates the main components and the direction of data flow. The recorded video is read frame by frame. YOLO detects vehicles in each frame and returns bounding boxes with confidence values. The occupancy-estimation component compares these detections with the parking regions loaded from JSON configuration files. The system then calculates floor-wise occupied and available counts and sends the results to the visualisation layer.

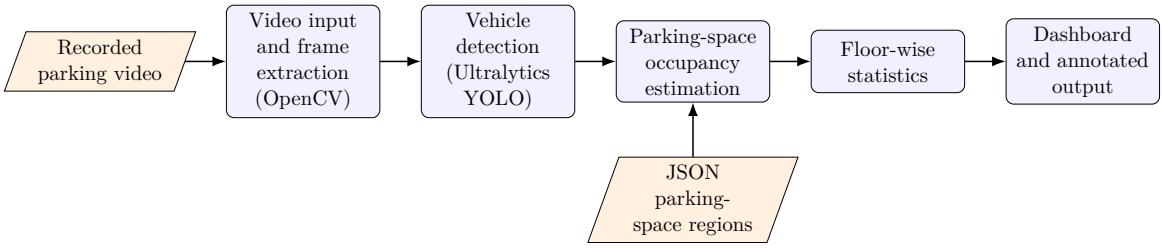


Figure 4.1: System architecture of the Smart Mall Parking Management System

4.2.1 Architecture Components

1. **Video input:** Opens a selected recorded parking video and supplies frames for processing.
2. **Frame processing:** Uses OpenCV to read, prepare, and display video frames.
3. **Vehicle detection:** Uses the configured YOLO model to detect vehicles and produce bounding boxes and confidence scores.
4. **Parking-space configuration:** Loads parking-space region coordinates from JSON files for the selected floor or video view.
5. **Occupancy estimation:** Compares detected vehicle locations with configured parking-space regions. The result is an estimated occupied/available state for each configured space.
6. **Statistics and presentation:** Aggregates the estimated states into floor-wise counts and displays the annotated output and dashboard information.

4.3 System Workflow

The processing workflow starts when the user selects a configured floor or its corresponding recorded video. The application loads the model and the relevant parking-space configuration. Each frame is passed to the detection model, and the returned vehicle detections are compared with the configured parking regions. A stability mechanism may be used by the implementation to reduce rapid state changes between successive frames. The system updates occupancy estimates, calculates the counts, and refreshes the displayed output.

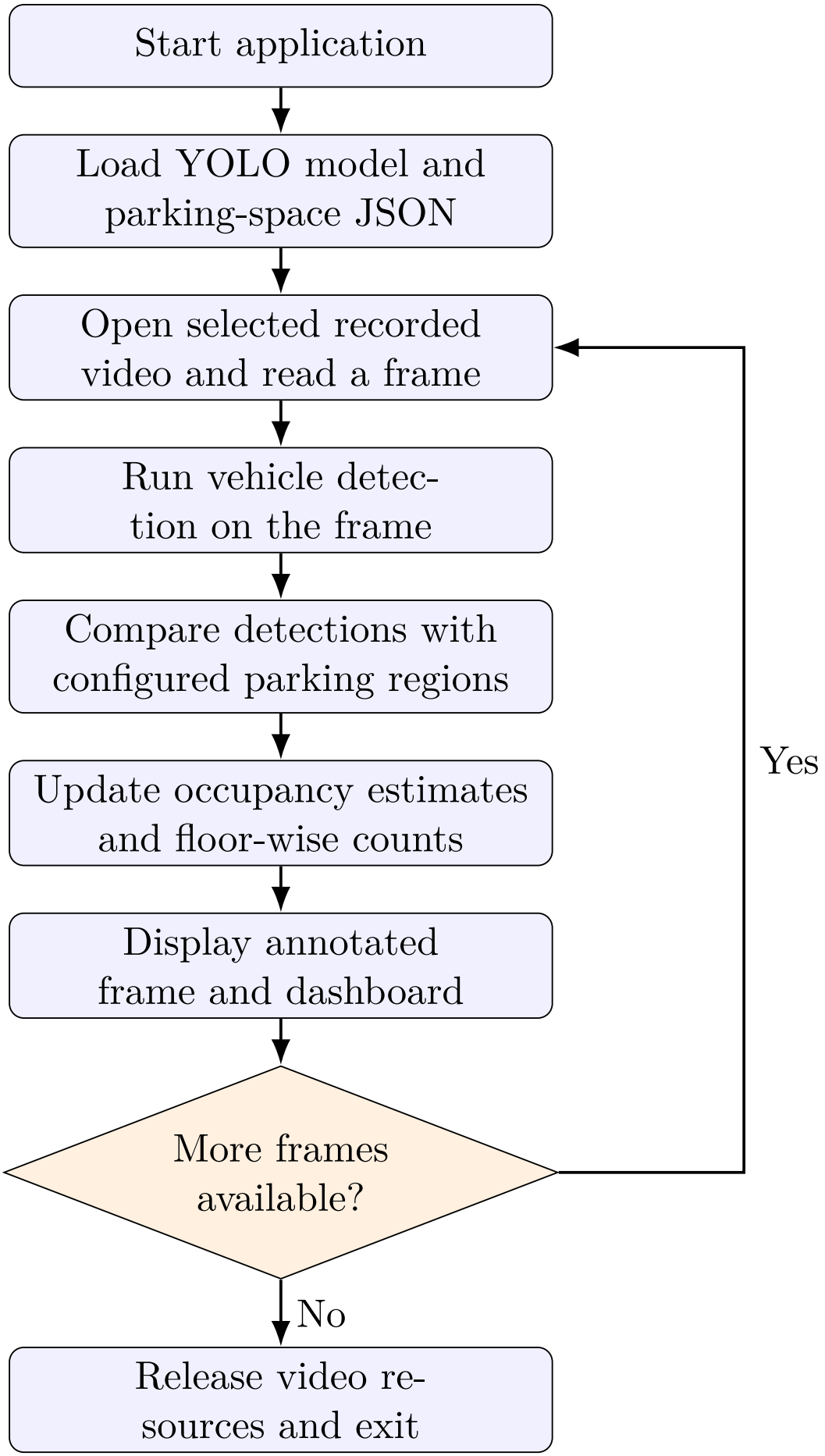


Figure 4.2: Processing workflow for parking occupancy estimation

4.4 Data Flow

The main data flow can be summarised as follows:

1. The recorded video provides image frames to the application.
2. The YOLO model processes a frame and returns vehicle detections.
3. The configuration loader provides the parking-space coordinates for the selected view.
4. The occupancy-estimation component associates vehicle detections with parking-space regions using the implemented geometric overlap and/or point-in-region checks.
5. The per-space state mechanism updates the estimated occupancy state.
6. The statistics component calculates occupied and available counts for the selected floor.
7. The application presents the annotated frame and the relevant information in the dashboard.

4.5 Parking-Space Configuration Design

Parking-space regions are stored in JSON configuration files. Each configured space should have a unique identifier and coordinates describing its region in the relevant video view. The coordinates must match the video resolution and camera viewpoint; otherwise, the detection-to-space association may be inaccurate. Separate configurations can be used for the different floor views. The exact JSON key names and file names in the final submission should match the actual project files.

4.6 Module Description

Table 4.1 summarises the major software modules and their responsibilities.

Table 4.1: System modules and responsibilities

Module	Responsibility
Video input	Reads frames from the selected recorded video.
Frame processing	Uses OpenCV to handle and display video frames.
YOLO detection	Detects vehicles and returns bounding boxes and confidence values.
Configuration loader	Loads parking-space regions from JSON configuration files.
Occupancy estimation	Compares vehicle detections with configured parking regions and estimates each space's state.
State stabilisation	Helps reduce rapid occupied/available changes across successive frames, according to the implemented logic.
Statistics	Calculates total, occupied, and available spaces for the selected floor.
Dashboard	Displays parking information and the system's available-space recommendation, where implemented.

4.7 Design Considerations and Limitations

The accuracy of occupancy estimates depends on camera viewpoint, video quality, lighting, occlusion, model detections, and the correctness of the configured parking regions. Region coordinates should be reviewed for each view. Since the current prototype processes recorded videos, it should not be described as a deployed live-CCTV service. The report should include measured performance values only when they have been obtained from actual experiments.

4.8 Summary

The system design separates video processing, vehicle detection, parking-space occupancy estimation, statistics calculation, and presentation. This modular organisation makes the prototype easier to understand, test, and maintain.

Chapter 5

Implementation

5.1 Development Environment

The project uses Python, OpenCV, Ultralytics YOLO, NumPy, and JSON configuration files. Record exact versions and the actual execution environment before submission.

5.2 Project Structure

The following is an example structure; update names to match your files.

```
SmartMallParking/
|-- main.py
|-- yolov8n.pt
|-- 1.mp4
|-- 2.mp4
|-- 3.mp4
|-- parking_spaces_1.json
|-- parking_spaces_2.json
|-- parking_spaces_3.json
```

5.3 Video Input

OpenCV reads frames from the configured video file. The application should check whether the video opened successfully.

Listing 5.1: Example of opening a video

```
import cv2
cap = cv2.VideoCapture("1.mp4")
if not cap.isOpened():
    raise RuntimeError("Unable to open video file")
```

5.4 Vehicle Detection

The YOLO model is applied to each frame. The following example is illustrative; use the parameters from the actual code.

Listing 5.2: Example YOLO inference call

```
results = model.predict(
    frame,
    classes=[2],
    conf=0.35,
    iou=0.45,
    verbose=False
)
```

For a standard COCO-pretrained YOLO model, class identifier 2 represents a car. Verify the class mapping if a different model is used.

5.5 Parking-Space Configuration

The program reads parking-space identifiers and coordinates from JSON. The regions must match the viewpoint and dimensions of each video.

5.6 Occupancy Estimation

The system compares detected car boxes with configured parking regions. Intersection-over-union and point-in-polygon checks help determine whether a vehicle corresponds to a space. The status-stability mechanism follows the threshold and counter logic implemented in the code.

5.7 Statistics Calculation

The system counts configured spaces and spaces estimated as occupied. Available spaces equal total spaces minus occupied spaces. The values are supplied to the dashboard.

5.8 Parking Recommendation

The recommendation function selects an available space according to the selection order in the implementation. It is a basic recommendation and does not calculate an optimal route.

5.9 Dashboard

The dashboard displays total, occupied, and available counts for each configured floor, plus an available-space recommendation when one is found. View-switching behaviour should be described according to the actual controls.

5.10 User Interface

The following screenshots show the parking visualisations for the three configured floors and the dashboard interface. The image named `4th Floor` is used as the dashboard screenshot; it is not treated as a fourth parking floor.

5.10.1 First Floor Parking Visualisation

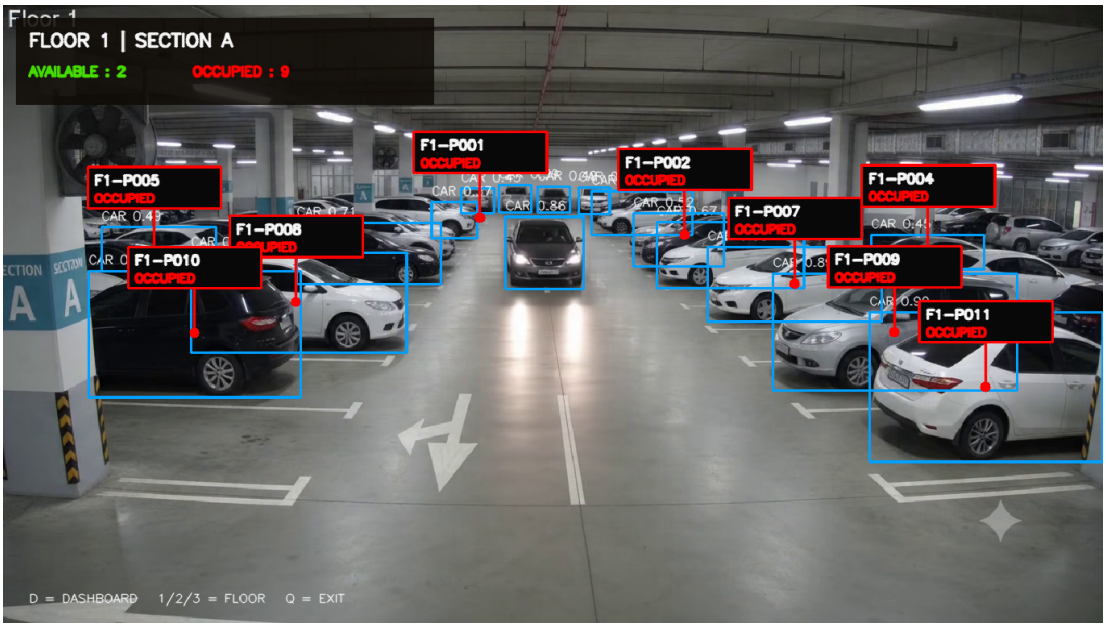


Figure 5.1: Parking Occupancy Detection on the First Floor

5.10.2 Second Floor Parking Visualisation

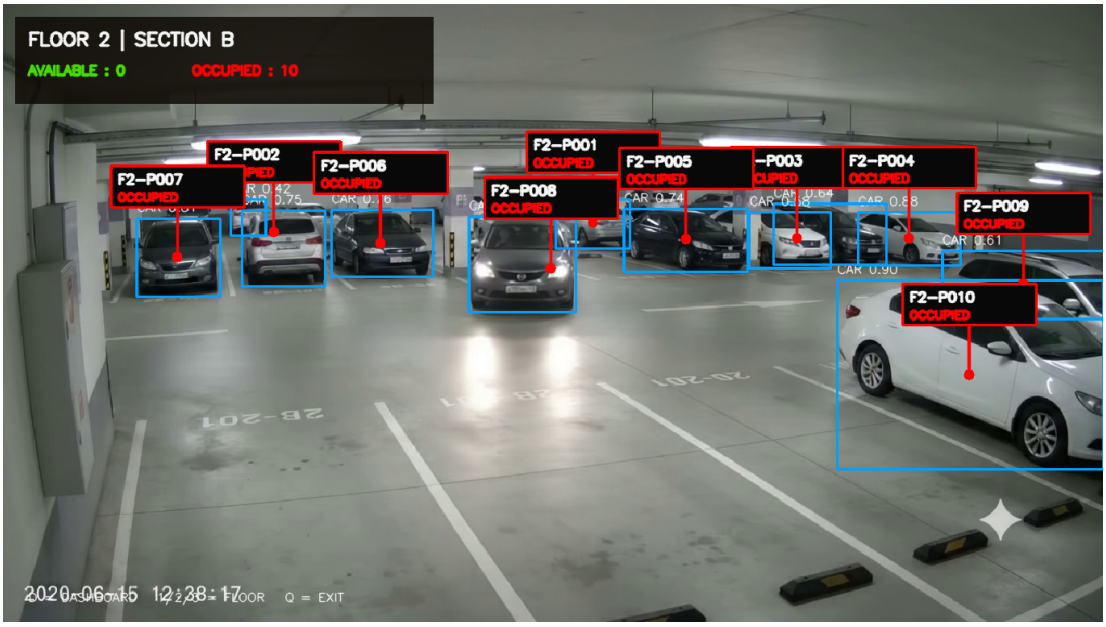


Figure 5.2: Parking Occupancy Detection on the Second Floor

5.10.3 Third Floor Parking Visualisation

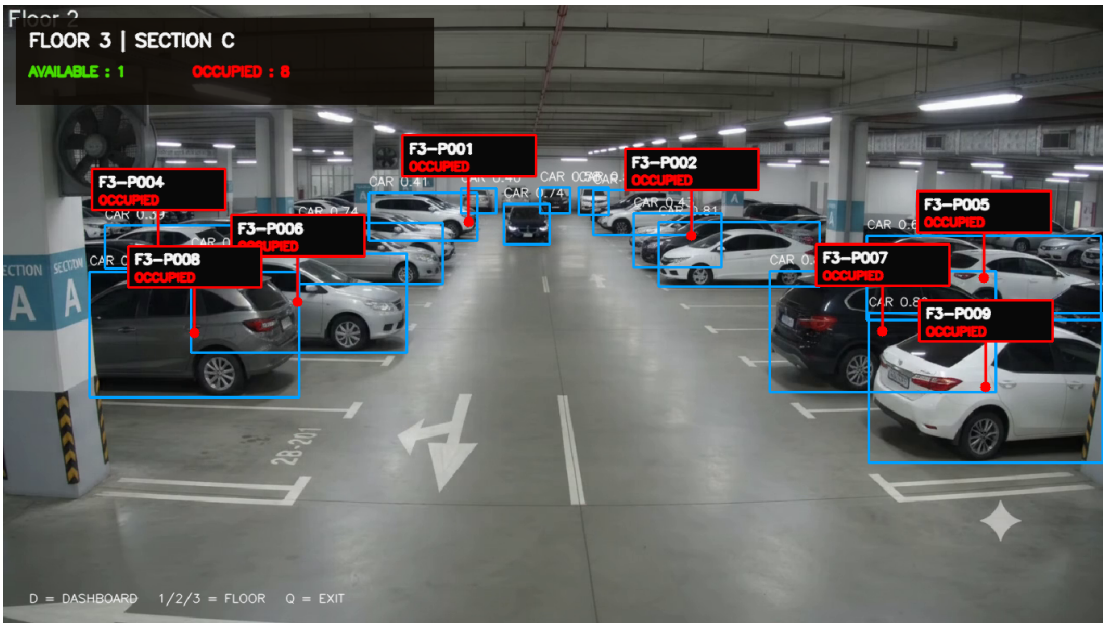


Figure 5.3: Parking Occupancy Detection on the Third Floor

5.10.4 Parking Management Dashboard

The dashboard presents the parking information and estimated availability generated by the prototype.

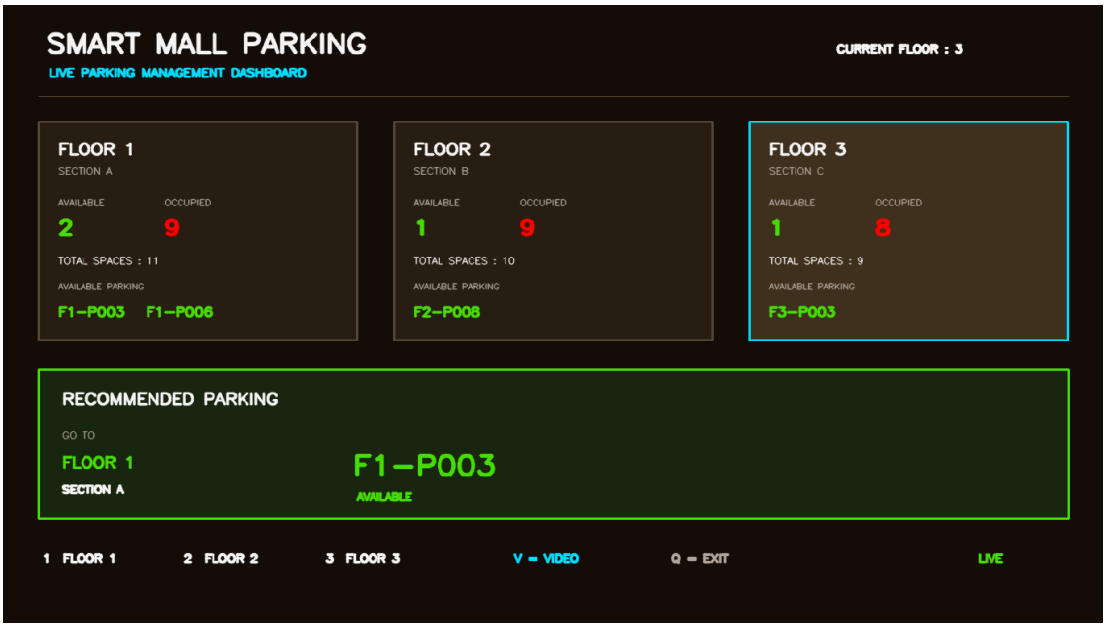


Figure 5.4: Smart Mall Parking Management System Dashboard

5.11 Integration

The main application connects model inference, video reading, configuration loading, occupancy estimation, statistics, and dashboard display. Missing files should be reported clearly.

5.12 Security and Privacy

The current prototype uses recorded videos. Any future use of live CCTV must use authorised access and follow the property owner's policies and applicable privacy requirements.

5.13 Summary

This chapter described the development environment, video input, detection, configuration, occupancy estimation, recommendation, and dashboard. Replace illustrative examples with the actual implementation before submission.

Chapter 6

Results and Discussion

6.1 Experimental Setup

Test the application with the actual recorded videos, JSON files, and YOLO weights. Record the machine configuration, model version, detection thresholds, and other parameters needed to reproduce the test.

6.2 Test Cases

Complete the actual-result and status columns after running each test.

Table 6.1: Functional test cases

ID	Test	Expected result	Actual result	Status
T1	Load model	Model loads without error	[Record result]	[Pass/Fail]
T2	Load configuration	Parking regions are loaded	[Record result]	[Pass/Fail]
T3	Open video	Frames are read successfully	[Record result]	[Pass/Fail]
T4	Detect cars	Car detections appear	[Record result]	[Pass/Fail]
T5	Estimate occupancy	Status is calculated for spaces	[Record result]	[Pass/Fail]
T6	Calculate counts	Occupied and available counts appear	[Record result]	[Pass/Fail]
T7	Open dashboard	Floor statistics are visible	[Record result]	[Pass/Fail]
T8	Switch views and exit	Controls work as intended	[Record result]	[Pass/Fail]

6.3 Observed Results

The prototype is designed to display car detections over recorded video and estimate occupancy from configured regions. Insert screenshots from actual execution and describe the observations. Do not claim a test passed unless it has been run.

6.4 Quantitative Evaluation

If labelled occupancy data is available, compare predicted status with ground truth. For binary classification:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \times 100\%.$$

Here TP, TN, FP, and FN denote true positives, true negatives, false positives, and false negatives. Define the positive class consistently. Do not report accuracy without labelled test observations. Processing speed may be measured as:

$$\text{FPS} = \frac{\text{Number of processed frames}}{\text{Elapsed time in seconds}}.$$

6.5 Discussion

Results depend on detection quality and the accuracy of parking-space configuration. Occlusion, poor lighting, camera angles, and inaccurate region boundaries may affect estimated status. The dashboard displays estimates and should not be treated as a guaranteed live inventory.

6.6 Limitations

The prototype processes recorded video, requires configured parking regions, may be affected by occlusion and lighting, uses a basic recommendation strategy, and has no established quantitative accuracy unless validated against labelled ground truth.

6.7 Summary

The final report should include actual screenshots, completed test cases, and measured performance values only.

Chapter 7

Conclusion and Future Scope

7.1 Conclusion

The Smart Mall Parking Management System demonstrates a prototype for estimating parking-space occupancy using Artificial Intelligence and Computer Vision. It uses Python, Ultralytics YOLO, OpenCV, and JSON-based parking-space configurations to process recorded parking videos. The system detects cars, compares bounding boxes with predefined regions, estimates occupied and available spaces, calculates statistics for three configured floors, and displays the information through a dashboard. A simple recommendation function identifies an available space from the estimated occupancy data. The project provides practical experience in object detection, video processing, geometric region matching, occupancy estimation, and dashboard development. Reliability depends on video quality, parking-region configuration, and detection performance. Further testing with labelled data is required to quantify accuracy.

7.2 Limitations

The current implementation uses recorded videos and configured regions. It does not claim automatic live CCTV integration, production deployment, persistent database storage, or optimal route planning.

7.3 Future Scope

- Integrate authorised live camera feeds.
- Improve detection and occupancy validation under occlusion and changing lighting.
- Add persistent storage for occupancy history.
- Develop a web-based dashboard.
- Synchronise multiple camera feeds while managing duplicate detections.
- Improve recommendations using distance and accessibility criteria.

- Evaluate labelled videos and report measured accuracy, precision, recall, and processing speed.

7.4 Final Remarks

The project establishes a foundation for video-based parking monitoring. With further validation, authorised camera integration, and improved guidance logic, the prototype could be extended into a more comprehensive parking-management application.

Bibliography

[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[2] Ultralytics, "Ultralytics YOLO Documentation and Repository." Available: https://docs.ultralytics.com/. Accessed: [Enter access date].

[3] OpenCV, "OpenCV Documentation." Available: https://docs.opencv.org/. Accessed: [Enter access date].

[4] Python Software Foundation, "Python Documentation." Available: https://docs.python.org/3/. Accessed: [Enter access date].

Chapter A

Additional Project Information

A.1 Execution Checklist

- Confirm the YOLO model loads correctly.
- Confirm all video paths are correct.
- Confirm each JSON file contains the intended parking regions.
- Check detections against actual frames.
- Verify occupancy counts against observed examples.
- Confirm the dashboard opens and displays floor statistics.
- Use screenshots from the actual implementation.
- Complete test results and include only measured performance values.

A.2 Screenshots and Evidence

Add additional application screenshots, configuration examples, and execution evidence here if required by the department. Give every figure a caption and refer to it in the text.

A.3 Team Contributions

Describe each member's actual contribution to coding, configuration, testing, documentation, or evaluation. Ensure the statement reflects the work performed.